Name: Saai Vardhan D
Registration number : 192525205

CSA 0421 - OPERATING SYSTEM

Experiment 6

Construct a C program to implement pre-emptive priority scheduling algorithm.

Executable C code :

```
#include <stdio.h>

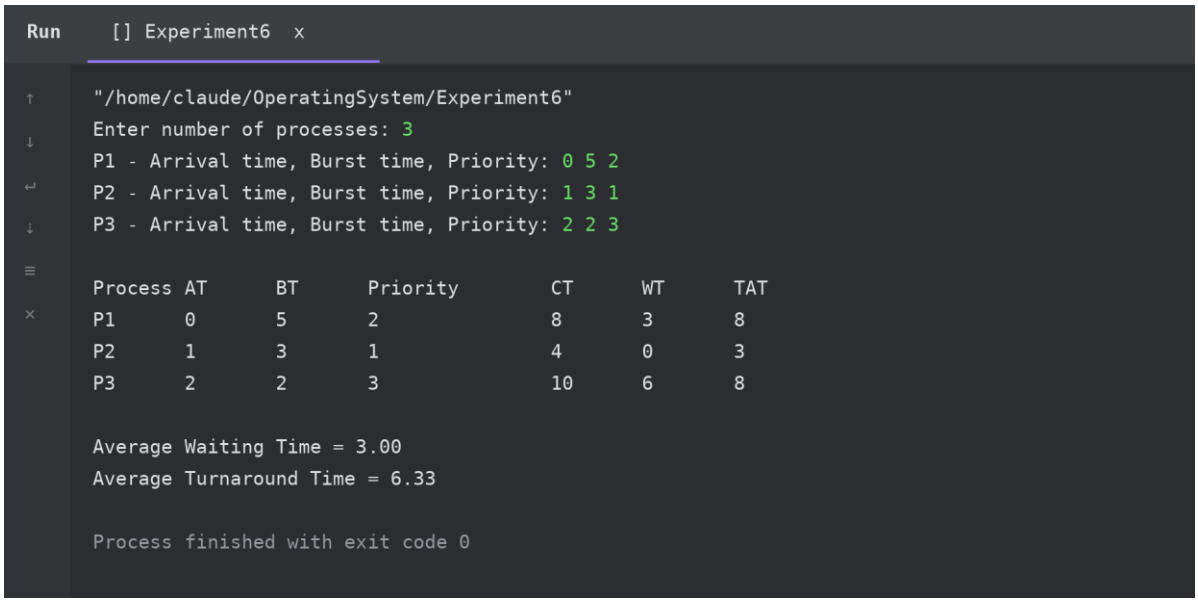
int main() {
    int n, i, time = 0, done = 0, hp, min;
    int at[10], bt[10], pr[10], rem[10], ct[10], wt[10], tat[10];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival time, Burst time, Priority: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rem[i] = bt[i];
    }

    // run one time unit at a time; always pick the highest priority (lowest
number)
    while (done < n) {
        hp = -1;
        min = 9999;
        for (i = 0; i < n; i++) {
            if (at[i] <= time && rem[i] > 0 && pr[i] < min) {
                min = pr[i];
                hp = i;
            }
        }
        if (hp == -1) {          // CPU idle
            time++;
            continue;
        }
        rem[hp]--;
        time++;
        if (rem[hp] == 0) {
            ct[hp] = time;
            done++;
        }
    }

    printf("\nProcess\tAT\tBT\tPriority\tCT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        avgwt += wt[i];
        avgtat += tat[i];
        printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\n", i + 1, at[i], bt[i], pr[i],
ct[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f\n", avgwt / n);
    printf("Average Turnaround Time = %.2f\n", avgtat / n);
    return 0;
}
```

Output Screenshot:



Experiment 7

Construct a C program to implement non-preemptive SJF algorithm.

Executable C code :

```
#include <stdio.h>

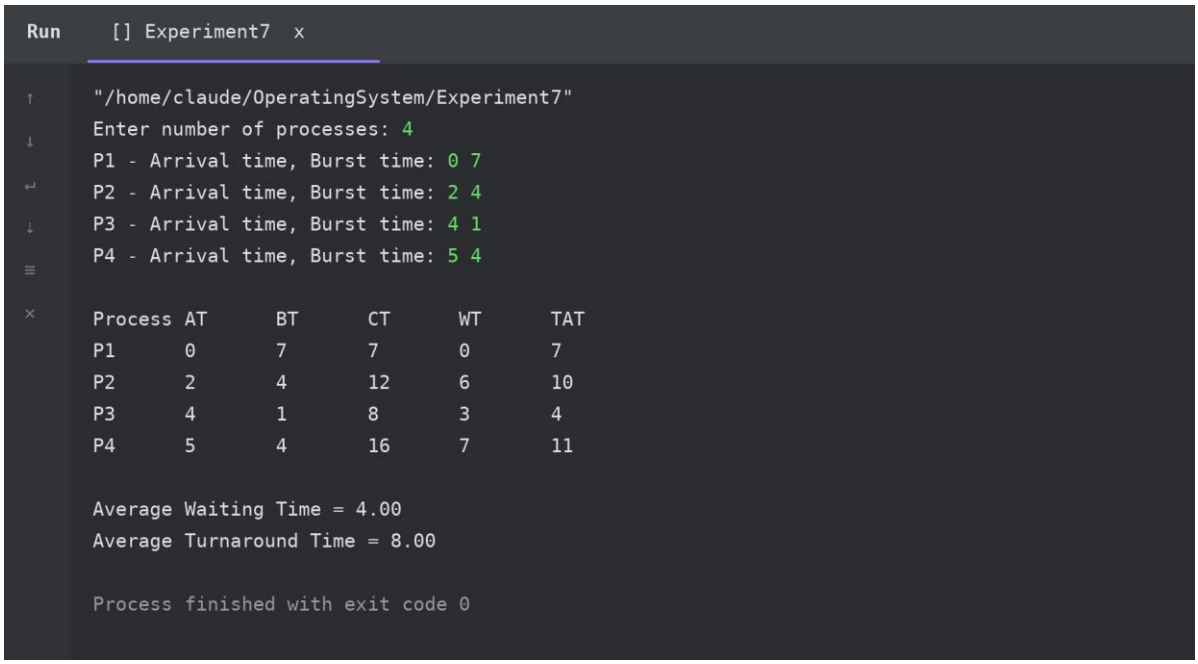
int main() {
    int n, i, time = 0, done = 0, sel, min;
    int at[10], bt[10], ct[10], wt[10], tat[10], flag[10] = {0};
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival time, Burst time: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    // once a process starts, it runs to completion (non-preemptive)
    while (done < n) {
        sel = -1;
        min = 9999;
        for (i = 0; i < n; i++) {
            if (!flag[i] && at[i] <= time && bt[i] < min) {
                min = bt[i];
                sel = i;
            }
        }
        if (sel == -1) {         // no process has arrived yet
            time++;
            continue;
        }
        time += bt[sel];
        ct[sel] = time;
        flag[sel] = 1;
        done++;
    }

    printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        avgwt += wt[i];
        avgtat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt[i],
tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f\n", avgwt / n);
    printf("Average Turnaround Time = %.2f\n", avgtat / n);
    return 0;
}
```

Output Screenshot:



Experiment 8

Construct a C program to simulate Round Robin scheduling algorithm with C.

Executable C code :

```
#include <stdio.h>

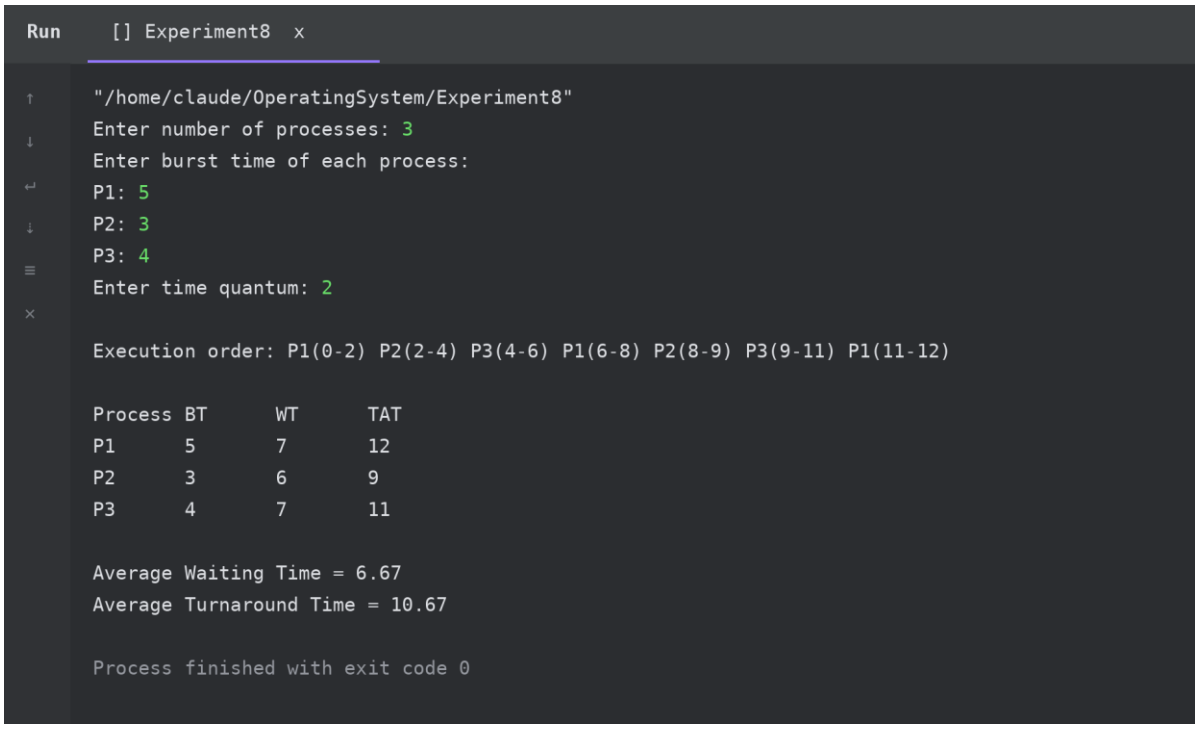
int main() {
    int n, i, q, t = 0, done;
    int bt[10], rem[10], wt[10], tat[10];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time of each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem[i] = bt[i];
    }
    printf("Enter time quantum: ");
    scanf("%d", &q);

    printf("\nExecution order: ");
    do {
        done = 1;
        for (i = 0; i < n; i++) {
            if (rem[i] > 0) {
                done = 0;
                int start = t;
                if (rem[i] > q) {
                    t += q;
                    rem[i] -= q;
                } else {
                    t += rem[i];
                    rem[i] = 0;
                    tat[i] = t;
                }
                printf("P%d(%d-%d) ", i + 1, start, t);
            }
        }
    } while (!done);

    printf("\n\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        wt[i] = tat[i] - bt[i];
        avgwt += wt[i];
        avgtat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f\n", avgwt / n);
    printf("Average Turnaround Time = %.2f\n", avgtat / n);
    return 0;
}
```

Output Screenshot:



Experiment 9

Illustrate the concept of inter-process communication using shared memory with a C program.

Executable C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>

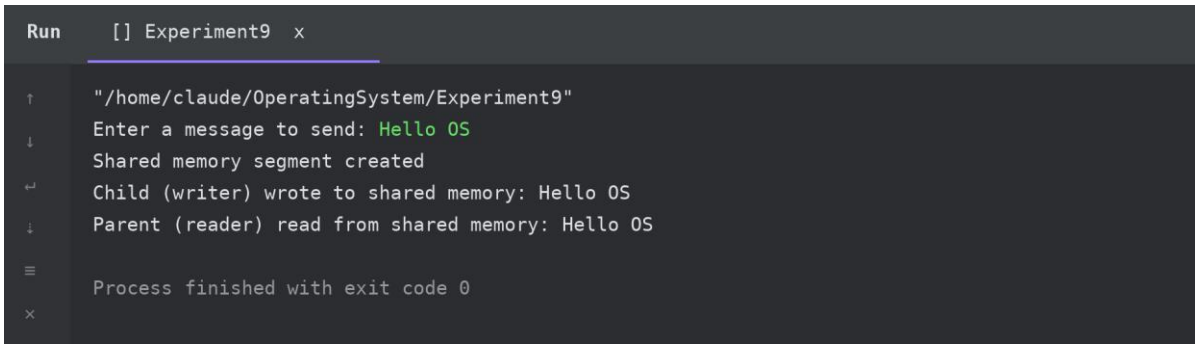
int main() {
    int shmid;
    char *shm, msg[100];
    pid_t pid;

    printf("Enter a message to send: ");
    fgets(msg, sizeof(msg), stdin);
    msg[strcspn(msg, "\n")] = '\0';

    // create a shared memory segment of 1024 bytes
    shmid = shmget(IPC_PRIVATE, 1024, IPC_CREAT | 0666);
    if (shmid < 0) {
        perror("shmget");
        exit(1);
    }
    printf("Shared memory segment created\n");

    fflush(stdout);
    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    } else if (pid == 0) {
        // child: attach and write
        shm = (char *)shmat(shmid, NULL, 0);
        strcpy(shm, msg);
        printf("Child (writer) wrote to shared memory: %s\n", shm);
        shmdt(shm);
        exit(0);
    } else {
        // parent: wait for the child, attach and read
        wait(NULL);
        shm = (char *)shmat(shmid, NULL, 0);
        printf("Parent (reader) read from shared memory: %s\n", shm);
        shmdt(shm);
        shmctl(shmid, IPC_RMID, NULL);   // remove the segment
    }
    return 0;
}
```

Output Screenshot:



Experiment 10

Illustrate the concept of inter-process communication using message queue with a C program.

Executable C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/wait.h>

struct message {
    long mtype;
    char mtext[100];
};

int main() {
    int msgid;
    struct message msg;
    char text[100];
    pid_t pid;

    printf("Enter a message to send: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = '\0';

    msgid = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);
    if (msgid < 0) {
        perror("msgget");
        exit(1);
    }

    fflush(stdout);
    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    } else if (pid == 0) {
        // child: receive the message
        msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0);
        printf("Child (receiver) got the message: %s\n", msg.mtext);
        exit(0);
    } else {
        // parent: send the message
        printf("Parent (sender) is sending the message...\n");
        fflush(stdout);
        msg.mtype = 1;
        strcpy(msg.mtext, text);
        msgsnd(msgid, &msg, strlen(text) + 1, 0);
        wait(NULL);
        msgctl(msgid, IPC_RMID, NULL);   // remove the queue
    }
    return 0;
}
```

Output Screenshot:

